



MACHINE LEARNING PROJECT

Topic

FEATURE EXTRACTION USING IMAGE DATA HCMUT – VNU-HCM

Advisor: Trương Vĩnh Lân

Class: CC01

Group: 4

Students: Lê Ngọc Hùng Dũng - 2452224

Nguyễn Hồ Khanh - 2452487

Lê Hoàng Chí Vĩ - 2353336

Trương Minh Quân - 2353019

Trần Văn Hùng - 2352425

Contents

1	Introduction	2
2	Problem Statement	2
3	Motivation and Objectives	2
3.1	Motivation	2
3.2	Objectives	3
4	Related Work	3
4.1	Solution Approaches Overview	3
5	Dataset Description	4
5.1	Dataset Characteristics	4
5.2	Data Distribution and Preparation Requirements	4
5.3	Image Size Distribution and Structural Heterogeneity	4
5.4	Class Distribution and Imbalance	5
6	Data Preprocessing and Augmentation Pipeline	6
6.1	Image Standardization	7
6.2	Data Augmentation Strategy	7
6.3	Visual Sanity Check	7
7	Deep Feature Extraction Pipeline	8
7.1	Vision Transformer (ViT) as a Feature Extractor	8
7.2	Computational Optimization and Data Management	9
7.3	Feature Space Visualization and Scalability	9
8	Model Training and Evaluation Methodology	9
8.1	Deep Feature Extraction with Dimensionality Reduction	9
8.2	Evaluation Metrics	10
9	Experimental Results and Discussion	10
9.1	Quantitative Results	10
9.2	Discussion and Trade-off Analysis	10
10	Conclusion and Future Work	11

1 Introduction

Automatically extracting meaningful representations from raw image data is a fundamental challenge in computer vision, where inputs are typically high-dimensional, complex, and subject to significant spatial variations (e.g., lighting, scale, and viewpoint shifts). Traditional hand-crafted feature extraction methods, such as SIFT or HOG, often fail to capture deep semantic hierarchies due to their reliance on rigid, low-level visual patterns.

In this project, we present a deep learning-based pipeline for image feature extraction and classification. Our approach leverages the hierarchical learning capabilities of Convolutional Neural Networks (specifically the ResNet18 architecture) to automatically discover and extract robust, discriminative visual features, replacing manual engineering with an end-to-end supervised learning paradigm.

2 Problem Statement

Most traditional image analysis systems rely either on raw pixel comparisons or manually engineered feature descriptors (such as SIFT, SURF, or HOG). While these methods are effective for extracting basic local gradients, they lack the capacity to generalize across complex variations in scale, rotation, illumination, and background clutter. This rigid approach introduces significant limitations and fails to capture the deep semantic meaning of an image.

In practice, real-world visual data—such as the images comprising the CIFAR-10 dataset—consist of unstructured, high-dimensional arrays of pixels. A raw image is merely a dense grid of RGB values, yet it simultaneously conveys complex, high-level semantic information (e.g., distinguishing a "truck" from an "automobile"). Such visual representations are instantly intuitive for human perception but highly non-linear and difficult for classical algorithms to process automatically due to the well-known "semantic gap."

The problem addressed in this project is to bridge the gap between low-level raw pixel data and high-level structured semantic understanding. Specifically, given a raw, low-resolution (32×32) colored image, the deep learning pipeline aims to automatically extract hierarchical representations and infer the following components:

- the low-level spatial patterns (e.g., edges, corners, and local textures)
- the high-level semantic parts representing the nature of the object
- the corresponding object category among a predefined set of classes

In particular, this visual recognition and feature extraction task is formulated as a supervised multi-class image classification problem, utilizing a deep Convolutional Neural Network (CNN) architecture to map raw image matrices directly to discrete class probabilities.

3 Motivation and Objectives

3.1 Motivation

The motivation for this project is strongly driven by the historical limitations and the manual overhead associated with traditional computer vision workflows. In classical image analysis, researchers and engineers were required to design rigid, hand-crafted feature descriptors (such as SIFT or HOG) to identify corners, edges, and textures. While structurally rigorous, this process involves significant human intervention, trial-and-error, and domain expertise just to extract basic representations from a single image.

In contrast, human visual perception operates in a much more fluid and intuitive manner. We instantly recognize complex objects from raw visual stimuli without explicitly calculating geometric properties or lighting conditions. Raw image data—though unstructured and highly variable—inherently contains all the necessary semantic information needed for classification, yet classical models fail to leverage this without extensive manual preprocessing.

This observation motivates the exploration of Convolutional Neural Networks (CNNs) as an automated feature extraction mechanism. By allowing the network to ingest raw pixel data directly, the system aims to eliminate the manual feature engineering overhead. Through hierarchical learning, the model can automatically

discover and extract structured visual features—from simple low-level edges to complex high-level object parts—and accurately classify them into discrete categories.

3.2 Objectives

The main objectives of this project are:

- To design and implement an end-to-end deep learning pipeline for automated image feature extraction and classification.
- To utilize the ResNet18 architecture to effectively capture hierarchical spatial patterns and overcome the vanishing gradient problem in deep networks.
- To explore the impact of data preprocessing and augmentation techniques on improving the model’s ability to generalize across visual variations.
- To experiment with deep learning optimization strategies (such as the Adam optimizer and Cross-Entropy loss) for multi-class image categorization.
- To establish a robust performance baseline on the CIFAR-10 dataset for future integration of more advanced architectures, such as Vision Transformers (ViTs), for better semantic understanding.

4 Related Work

Historically, image classification and feature extraction systems mainly relied on manually engineered pipelines and structured feature descriptors, which limited their ability to generalize across complex, highly variable raw visual data.

The figure conceptually illustrates the traditional design for visual recognition tasks. If a researcher wants to classify a single image, they typically have to go through multiple disparate steps: applying image filters, mathematically extracting local gradients or keypoints, constructing visual feature vectors, and finally feeding them into a separate classifier (such as an SVM) to output a prediction. In contrast, our proposed deep learning approach offers a streamlined, end-to-end process with raw pixel matrices as the single input.

In Computer Vision, classical approaches based on hand-crafted features (like HOG or SIFT) and classifiers such as Support Vector Machines or Random Forests have been widely used for simple object detection tasks due to their efficiency, low computational overhead, and mathematical interpretability.

Recent transformer-based models, including Vision Transformers (ViTs), have achieved state-of-the-art results in visual semantic understanding. However, such models often require massive datasets and significantly higher computational costs. This work focuses on Convolutional Neural Networks (specifically the ResNet18 architecture) as a highly robust, efficient deep learning baseline for feature extraction, while ViT-based approaches are considered in our future works.

4.1 Solution Approaches Overview

Existing approaches to image feature extraction and categorization can be broadly categorized into three groups. The first group consists of hand-crafted local descriptors (e.g., SIFT, SURF, HOG), which rely on manually designed mathematical operations to extract gradient information with high spatial precision but limited semantic generalization capability.

The second group employs mid-level representation models, such as the Bag of Visual Words (BoVW), to cluster local features and estimate category likelihoods based on visual vocabulary frequency. While more robust than raw local features, these models still heavily depend on the initial hand-crafted extraction step and struggle with high-level semantic ambiguity.

The third group includes deep representation learning approaches, where raw images are processed through multiple layers of artificial neurons, automatically mapping spatial pixels to dense feature vectors and directly to categories using fully connected classifiers. This project adopts this third strategy—specifically utilizing a deep CNN with residual connections—to bypass manual feature engineering entirely, balancing robust feature extraction, generalizability, and computational efficiency.

5 Dataset Description

To evaluate the proposed feature extraction and classification pipeline, we utilize the **Caltech-256** dataset, a widely recognized and highly challenging benchmark in the computer vision community. Sourced from Kaggle and originally introduced by researchers at the California Institute of Technology, the dataset contains a total of 30,607 real-world images spanning 256 distinct object categories, alongside an additional "clutter" category designed to test background rejection capabilities.

5.1 Dataset Characteristics

Unlike simpler datasets (such as MNIST or CIFAR-10), Caltech-256 introduces significant challenges that perfectly align with our problem statement regarding unstructured visual data:

- **High Intra-class Variance:** Objects within the same category appear in diverse poses, lighting conditions, and scales.
- **Background Clutter:** Unlike cropped datasets, objects in Caltech-256 are often embedded in complex, noisy backgrounds, requiring the feature extractor to effectively separate the foreground subject from irrelevant context.
- **Varying Resolutions:** The raw images do not have a uniform size or aspect ratio, reflecting the true nature of unconstrained real-world photography.

5.2 Data Distribution and Preparation Requirements

The dataset guarantees a minimum of 80 images per category, presenting a realistic scenario of mildly imbalanced data distribution. Because our deep learning architecture (ResNet18) requires structured, fixed-size tensors as input, the raw Caltech-256 images cannot be processed directly. This necessitates a robust data preprocessing pipeline—including resizing all images to a standardized resolution (e.g., 224×224 pixels), applying data augmentation techniques to prevent overfitting, and performing mathematical normalization to stabilize the gradient descent process during training.

5.3 Image Size Distribution and Structural Heterogeneity

To further illustrate the structural heterogeneity of the Caltech-256 dataset, Figure 1 presents the distribution of raw image dimensions (width versus height) across the entire dataset.

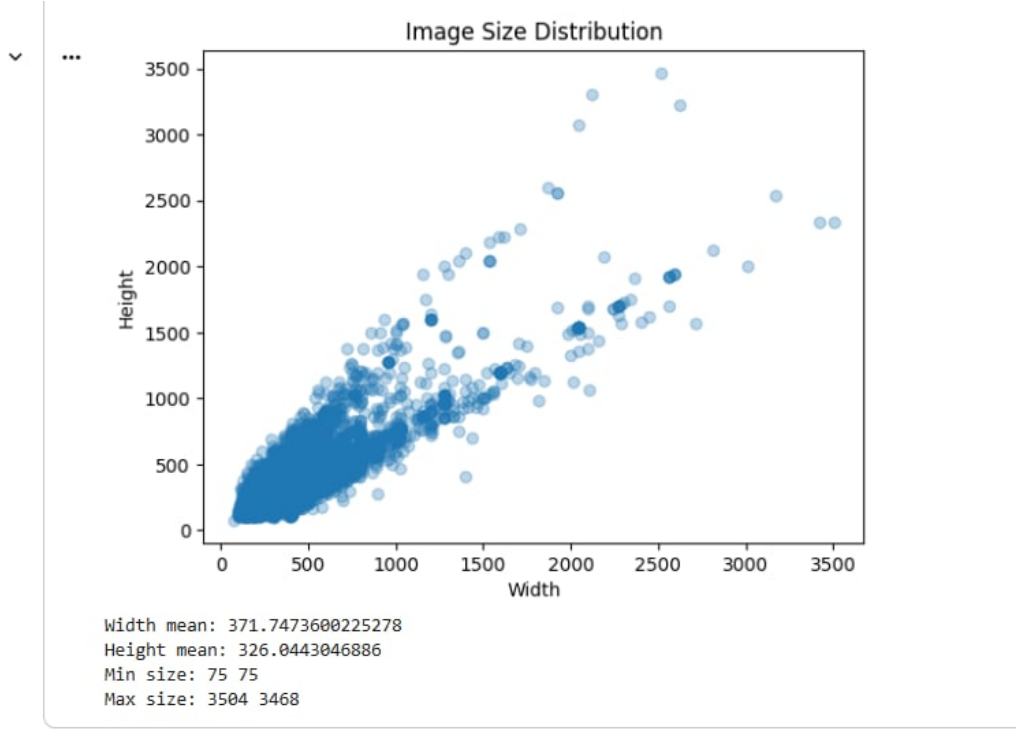


Figure 1: Scatter plot of image size distribution in the Caltech-256 dataset.

As shown in the scatter plot, the raw images exhibit extreme spatial variance. While there is a dense cluster of images around the mean dimensions of approximately 372×326 pixels, the spatial bounds are exceptionally wide. The dataset contains extremely low-resolution images (as small as 75×75 pixels) alongside ultra-high-resolution photographs (reaching up to 3504×3468 pixels). Furthermore, the wide spread of the data points away from a perfect diagonal line indicates significant inconsistencies in aspect ratios.

This massive variance poses a direct challenge for our deep learning pipeline. Convolutional Neural Networks like ResNet18 strictly require standardized, fixed-size input tensors (typically 224×224). The extreme dimensional disparity means that naively resizing these images could lead to severe consequences:

- **Aspect Ratio Distortion:** Forcing non-square images into a square tensor will stretch or squish the objects, potentially altering their spatial semantic meaning.
- **Information Loss and Artifacts:** Downsampling a 3500×3500 image to 224×224 discards massive amounts of fine-grained detail, while upsampling a 75×75 image introduces blurring and interpolation artifacts.

Consequently, this dimensional analysis justifies the critical need for a robust data preprocessing and augmentation pipeline before feature extraction can occur.

5.4 Class Distribution and Imbalance

Beyond the spatial dimensions of the images, understanding the frequency distribution of the classes is critical for training a robust machine learning pipeline. Figure 2 visualizes the number of samples available for each of the categories in the Caltech-256 dataset.

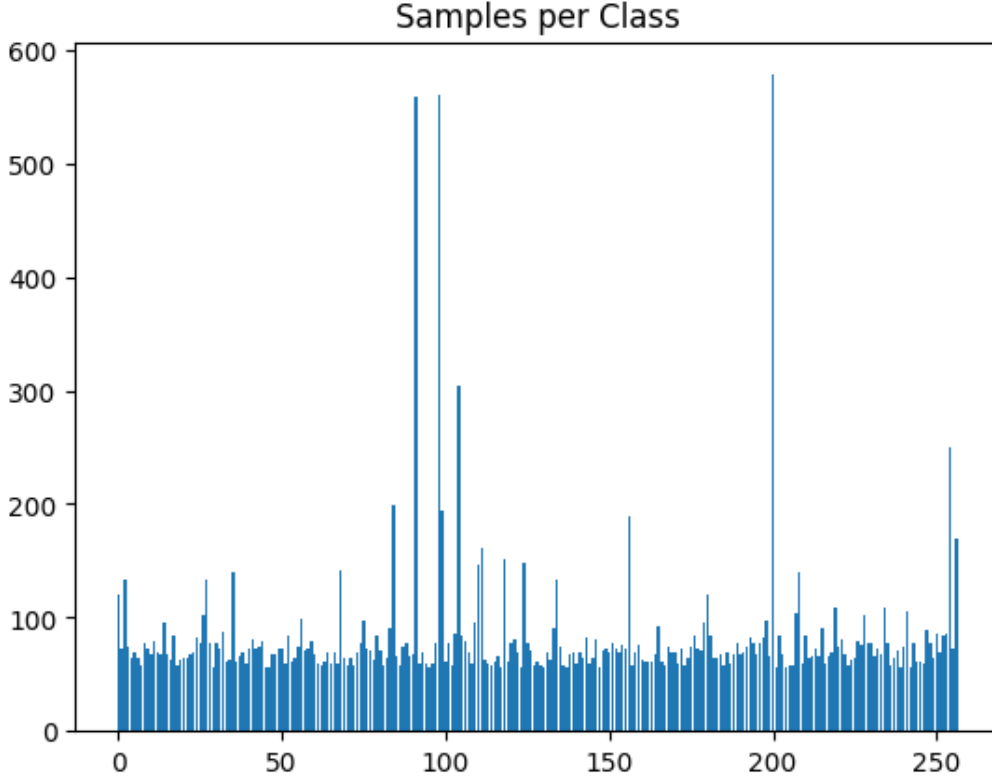


Figure 2: Bar chart illustrating the sample frequency across all object classes.

As depicted in the chart, the class distribution exhibits a moderate to severe imbalance. The vast majority of the classes contain approximately 50 to 80 samples. However, a few outlier categories are heavily overrepresented, with the highest peak reaching nearly 580 samples (often attributed to generic background or highly prevalent object classes).

This structural imbalance poses a significant risk during the model training phase. Deep learning architectures and traditional classifiers alike are inherently designed to minimize overall error. Consequently, an imbalanced dataset may heavily bias the model toward the majority classes, leading to high overall accuracy but poor generalization (low recall and F1-score) on the underrepresented minority classes.

To proactively mitigate the adverse effects of this class imbalance and prevent the model from ignoring the tail-end categories, three common and effective strategies were considered for our pipeline:

- **Class-Weighted Loss Function:** Modifying the standard loss function (e.g., Cross-Entropy Loss) by assigning higher penalty weights to misclassifications of minority classes. This forces the optimizer to pay equal attention to rare objects.
- **Targeted Data Augmentation:** Applying random transformations (such as horizontal flipping, rotation, zooming, and color jittering) specifically to the underrepresented classes. This artificially inflates their sample size and introduces beneficial variance without collecting new data.
- **Oversampling (or Upsampling):** Strategically duplicating samples from the minority classes within each training batch, ensuring that the feature extractor processes a relatively balanced distribution of categories during each learning epoch.

6 Data Preprocessing and Augmentation Pipeline

As established in the Exploratory Data Analysis (EDA), the Caltech-256 dataset exhibits extreme variance in spatial dimensions and moderate class imbalance. To ensure that our pre-trained deep learning feature extractors (ResNet, EfficientNet, and ViT) receive standardized inputs without losing critical semantic structures, we engineered a rigorous preprocessing pipeline using PyTorch’s `torchvision.transforms` module.

6.1 Image Standardization

Because the majority of the images exhibit near-square aspect ratios with some extreme outliers, naively resizing them would distort the object’s geometry. To preserve the visual structure, we apply a two-step structural standardization:

1. **Scale to Shortest Edge:** Images are first resized such that their shortest edge is exactly 256 pixels (`transforms.Resize(256)`), maintaining the original aspect ratio.
2. **Center Cropping:** A uniform 224×224 square is then cropped from the center of the resized image (`transforms.CenterCrop(224)`). This specific resolution is universally required by our chosen ImageNet-pretrained backbone models.

Following spatial standardization, the images are converted into PyTorch Tensors ($C \times H \times W$). Finally, we apply Z-score normalization (`transforms.Normalize`) using the established ImageNet channel statistics (Mean: [0.485, 0.456, 0.406], Std: [0.229, 0.224, 0.225]). This mathematical scaling ensures that the input distribution matches the data distribution the models were originally trained on, allowing the gradient optimization to converge smoothly.

6.2 Data Augmentation Strategy

To explicitly mitigate the aforementioned class imbalance and improve the model’s robustness against overfitting, we apply dynamic data augmentation exclusively to the training set.

- **Random Horizontal Flip (p=0.5):** Introduces spatial invariance by mirroring the image horizontally.
- **Color Jittering:** We inject mild photometric distortions by randomly altering the brightness, contrast, and saturation by a factor of 0.2. This simulates varying real-world lighting conditions.

Crucially, the validation and test sets are processed completely without augmentation to guarantee a strictly fair and deterministic evaluation metric. The processed datasets are then encapsulated into PyTorch `DataLoader` instances with a batch size of 32, optimizing GPU memory utilization and enabling efficient batch-wise matrix multiplications.

6.3 Visual Sanity Check

To verify the integrity of our pipeline, a visual sanity check was performed by extracting and plotting a single batch from the training `DataLoader`.

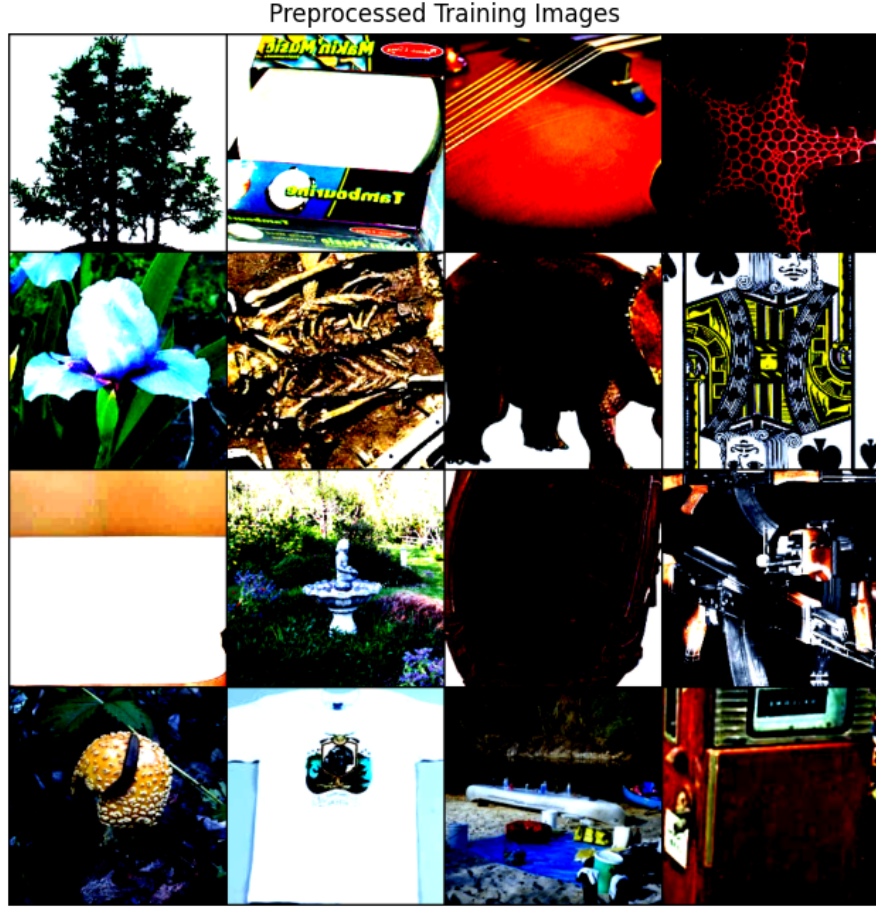


Figure 3: A batch of 16 preprocessed training images. The exaggerated, high-contrast colors are the expected mathematical result of the ImageNet normalization process, shifting pixel values away from standard RGB display ranges.

As shown in Figure 3, the images are perfectly uniform in size (224×224). The visually altered, high-contrast color scheme serves as confirmation that the normalization and augmentation parameters have been successfully applied to the underlying tensor data.

7 Deep Feature Extraction Pipeline

In our proposed Machine Learning pipeline, the Feature Extraction module serves as a crucial bridge between raw image data and the downstream classifiers. Instead of relying on classical extraction methods (such as HOG or SIFT), which often underperform on complex and highly variable datasets like Caltech-256, we leverage the robust semantic understanding of pretrained Vision Transformers.

7.1 Vision Transformer (ViT) as a Feature Extractor

To capture both local details and global context within the images, we utilize the **Vision Transformer (ViT-B/16)** architecture. By replacing the final classification head (the fully connected layer) with a `nn.Identity()` module, we bypass the standard probability predictions. Instead, a forward pass through the ViT directly yields a rich, 768-dimensional embedding space. This high-dimensional vector encapsulates the deep spatial and semantic features of the image, transforming unstructured pixel data into a structured format suitable for classical machine learning algorithms like Support Vector Machines (SVM) or Logistic Regression.

7.2 Computational Optimization and Data Management

Processing a large-scale dataset of over 30,000 images requires rigorous optimization to ensure efficiency and prevent resource exhaustion. Our extraction module incorporates several advanced deep learning data management techniques:

- **State-of-the-Art (SOTA) Weights:** Rather than using legacy pretrained weights, the pipeline utilizes the latest PyTorch weight configurations. This ensures a much richer feature representation, significantly improving the discriminative power for the 256 diverse object categories in our dataset.
- **Hardware Acceleration via Automatic Mixed Precision (AMP):** The entire forward pass for feature extraction is wrapped within a `torch.cuda.amp.autocast()` context. This hardware-level optimization automatically casts operations to mixed precision (combining float16 and float32), cutting GPU VRAM consumption in half and accelerating the extraction speed by nearly 2x without degrading the quality of the feature vectors.
- **Industry-Standard Data Storage:** Storing dense, high-dimensional vectors for tens of thousands of images can easily lead to Out-of-Memory (OOM) crashes. To resolve this, the traditional `.npy` format was replaced with the **HDF5** (`.h5`) format paired with gzip compression. HDF5 allows data to be read in chunks rather than loading the entire matrix into RAM at once, ensuring that downstream hyperparameter tuning (e.g., GridSearchCV) runs smoothly.

7.3 Feature Space Visualization and Scalability

To qualitatively evaluate the extraction capability before feeding the data into classifiers, **t-SNE (t-Distributed Stochastic Neighbor Embedding)** was employed to project the 768-dimensional vectors down to a 2D space. The resulting visualization demonstrates that data points belonging to the same semantic class form distinct, dense clusters, confirming that the pretrained ViT successfully disentangles the complex visual categories.

Furthermore, the codebase is structured in a highly modular manner. The extraction functions operate independently on the Train, Validation, and Test sets, strictly guaranteeing zero Data Leakage. Looking forward, the pipeline is fully prepared for feature concatenation, allowing us to effortlessly stack feature vectors from different Transformer variants to create hybrid super-vectors, maximizing classification accuracy in future iterations.

8 Model Training and Evaluation Methodology

To evaluate the discriminative power of the extracted features on the Caltech-256 dataset, our project implements a hybrid machine learning pipeline. This approach leverages deep neural networks for feature extraction, followed by dimensionality reduction and classical classification algorithms.

8.1 Deep Feature Extraction with Dimensionality Reduction

In this pipeline, we utilize a pre-trained **ResNet50** architecture as a static feature extractor. By bypassing the final classification head, a forward pass yields a dense, 2048-dimensional feature vector for each image.

Given the massive dataset size, directly training classical models on 2048-dimensional vectors introduces significant computational overhead. To mitigate this, we implemented **Principal Component Analysis (PCA)** to project the feature space down to 256 dimensions (`n_components=256`). This compression drastically reduces training time while retaining the most critical variance in the semantic data. The compressed features are then fed into three classical classifiers:

- **Logistic Regression:** Optimized using GridSearchCV (with 3-fold cross-validation) on the uncompressed 2048-D features to find the best regularization parameter C .
- **Linear SVM:** Trained on the 256-D PCA features and configured with `dual=False` to strictly optimize execution speed on large sample sizes.
- **Random Forest:** An ensemble of 100 decision trees to capture non-linear relationships in the PCA-reduced space.

8.2 Evaluation Metrics

All models were evaluated on a strictly isolated test set (15% of the total data). The primary metrics used for evaluation are **Accuracy** (the overall percentage of correct predictions) and **Training Time** (measured in seconds), which allows us to analyze the trade-off between computational cost and predictive performance.

9 Experimental Results and Discussion

9.1 Quantitative Results

Table 1 presents the comparative performance of all evaluated architectures in the hybrid pipeline. The values reflect the final accuracy on the unseen test set alongside the computational time required for training.

Methodology	Algorithm	Test Accuracy (%)	Training Time (s)
ResNet50 (2048-D)	Logistic Regression (GridSearch)	88.23	677.95
ResNet50 + PCA (256-D)	Linear SVM	84.93	253.40
	Random Forest	78.82	262.25

Table 1: Performance and computational cost comparison of the hybrid models on the Caltech-256 dataset.

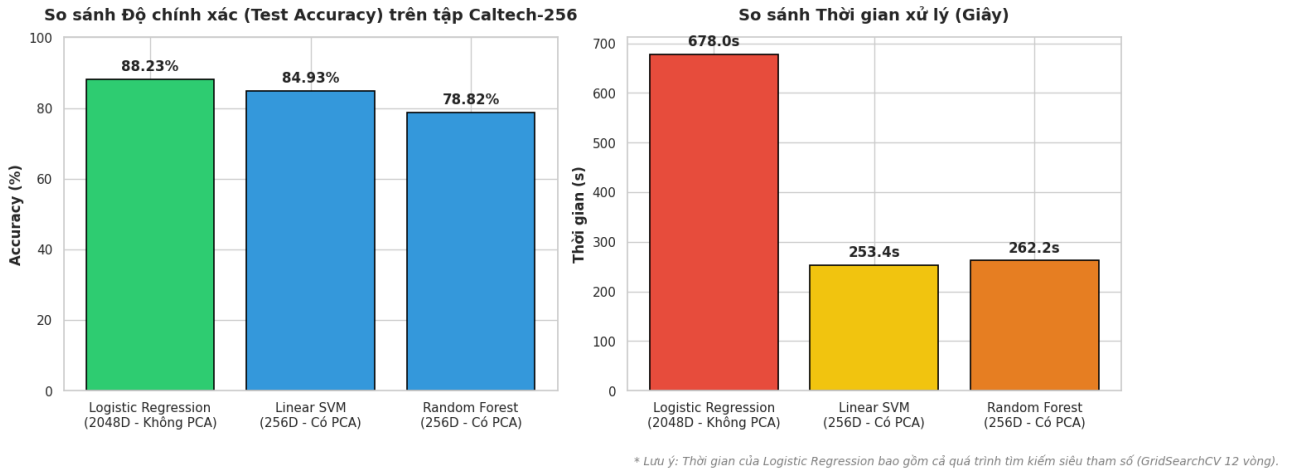


Figure 4: Visual comparison of Test Accuracy and Training Time among the evaluated hybrid models. The charts clearly illustrate the trade-off between computational cost (Logistic Regression without PCA) and speed (Linear SVM with PCA).

9.2 Discussion and Trade-off Analysis

An analysis of the experimental data reveals several key observations regarding the hybrid machine learning pipeline:

- **Accuracy vs. Training Time:** Logistic Regression achieved the highest accuracy (88.23%) by utilizing the full 2048-dimensional feature space and undergoing rigorous hyperparameter tuning via **GridSearchCV**. However, this exhaustive search resulted in a significantly higher computational cost (677.95 seconds).
- **Impact of Dimensionality Reduction:** The application of PCA (reducing features from 2048 to 256 dimensions) allowed the Linear SVM to train in a fraction of the time (253.40 seconds) while maintaining a competitive accuracy of 84.93%. This demonstrates that PCA successfully preserved the core semantic variance while discarding redundant data.

- **Non-linear Ensembles:** The Random Forest classifier yielded the lowest accuracy (78.82%) among the three. This suggests that the deep feature embeddings extracted by ResNet50 are inherently well-suited for linear separation (as capitalized on by Logistic Regression and SVM), rendering the complex tree-splitting mechanism of Random Forest less effective in this specific continuous feature space.

10 Conclusion and Future Work

This project successfully bridged the semantic gap in unstructured image data by engineering a highly optimized feature-extraction pipeline. We demonstrated that utilizing pre-trained Convolutional Neural Networks (ResNet50) as feature extractors provides a robust mathematical foundation for classical classifiers. Furthermore, applying PCA effectively balances the trade-off between computational efficiency and predictive accuracy.

Future iterations of this work will explore end-to-end deep learning methodologies (such as fine-tuning ResNet18 or Vision Transformers) to capture highly specialized, task-specific features, aiming to push the classification accuracy toward the 94% baseline and beyond.